DIDA Project Report: Simple implementation of a Replicatd Key-Value Store using the Paxos consensus Algorithm

Vasco Silva ist199132
Diogo Rodrigues ist1112192
José Vilar Gomes ist1100598
Instituto Superior Técnico
DIDA MEIC-T

Abstract

This report details the implementation of a distributed, transactional key-value store, that uses a variation of the Paxos consensus algorithm. This system consists of multiple servers that maintain the replicated state, transactional clients that execute simple read and update operations and a distinguished client which is responsible higher order admin-like operations. We followed the 4 steps described in the projects specifications, concluding our project with an implementation of reconfigurable Multi-Paxos.

1. Introduction

The project was divided into four key phases to gradually enhance the system's reliability and adaptability. Initially, we implemented an uncoordinated client-server model. We then introduced a fixed leader-based sequencer to manage request ordering. Subsequently, the basic Paxos algorithm was integrated to ensure consensus, followed by the implementation of reconfigurable Paxos, which allowed dynamic reconfiguration of server nodes. In the final step, we achieved reconfigurable Multi-Paxos, enabling concurrent transaction processing and more efficient configuration changes. This report will describe the specifications of how our final solution implements a version of a Reconfigurable, Multi-Paxos based replicated key-value store. Furthermore, the logic behind the choices that were made in this implementation, and the theory behind it was based on [1] [2] [3].

2. Server State Data Structures

The Server State is the primary class responsible for almost all functionality. It handles the processing and execution of the client requests as well as the execution of the paxos consensus algorithm, based on a configuration, to ensure consistent replication. The data structures explained ahead are what enable ServerState to efficiently manage client requests, maintain transaction order, and achieve consensus, ensuring that all nodes in the system maintain a consistent view of the replicated state.

2.1. Client Request Processing

For read requests, no consensus is required and these requests are answered immediately with the value present at the specified key in the key-value store. For incoming transactions, client requests are placed in a queue (**request_queue**), implemented as a *PriorityBlockingQueue*. This structure is chosen for its thread safety and its heap-based mechanism, which automatically reorders the queue based on priority when a request is added. Each entry is a `RequestQueueEntry` object, which contains the transaction request ID, an atomic integer representing the priority and the `TransactionRecord` itself. Incoming requests are also stored alongside a Java `CompletableFuture` in the **request_future_map**, to facilitate later responses to the client.

2.2. Handling Paxos State

To manage Paxos state effectively, we rely on three fundamental data structures:

- **Paxos Round State Map** - Stores objects that track critical Paxos state attributes for each round. These attributes include a lock to manage concurrent prepare or accept requests from different leaders, the leader's timestamp, the highest observed prepare timestamp, the highest recorded accept timestamp, and the last request accepted by the server in that Paxos round. This last element ensures that future leaders are informed of requests that were previously proposed and accepted

but not added to the transaction execution log due to a lack of majority.

- **Transaction Consensus Map** - Maps each transaction to its current state in the Paxos consensus process. Transactions can be marked as **AWAITING_PROPOSAL** (ready to be proposed by the leader), **PENDING_EXECUTION** (accepted in consensus but not yet executed), or as either **COMMITTED** or **ABORTED** if they have been finalized.

- **Transaction Execution Log** - An array that records the request IDs of transactions accepted by Paxos, with each transaction's consensus round represented by its index in the array.

2.3. Handling Configuration

We maintain all available configurations in a two-dimensional array that stores the server indices for each configuration. This array will be used in conjunction with the current configuration attribute to determine which configuration should be utilized for each Paxos consensus round. Phase one and phase two requests are only sent to servers that belong to the current configuration.

2.4. Transaction execution

In our implementation, we rely on a single execution thread responsible for executing transactions that have been added to the execution log after being accepted in consensus. For a transaction to be executed, it must satisfy two conditions: first, its predecessor in the log must have completed its execution; second, the transaction record associated with the request must have been received from the client.

The executor operates by searching the log, starting from the index of the last transaction it executed, until it encounters an empty index or a transaction that has been accepted but lacks an available transaction record for execution. It will then wait for either a transaction to be added to the empty index or for the request containing the transaction record to arrive from the client.

Upon executing a transaction the executor will set the request's state to **COMMITTED** or **ABORTED** and complete the client request future so that the client request is finalized.

3. Consensus Walkthrough

3.1. Leader

3.1.1 Preparing for Paxos

Once a server is elected as the leader it will then prepare to run Paxos.

- **Starting Index** - The leader first determines the index from which to begin running the Paxos consensus rounds by searching for the first missing consensus round in the transaction log.

- **Configuration** - To determine the configuration for executing the Paxos rounds, the leader loads the most recent configuration by backtracking through the transaction log from the starting index, leading to two possible scenarios:

  1. A regular request that has been executed is found before encountering an unexecuted reconfiguration request. In this case, reading the value of key 0 (which tracks the configuration) yields the most recent configuration.
  2. An unexecuted reconfiguration request is found before a regular request that has been executed. The configuration specified by this reconfiguration request will then be considered the most recent configuration.

- **Building a Request Batch** - To implement a variant of Multi-Paxos, we have opted to conduct consensus across multiple rounds simultaneously. To achieve this, the leader iteratively constructs a batch of requests for proposal by fetching them from the queue, constrained by the following factors:

  – The number of requests available in the request queue.
  – The maximum number of contiguous empty consensus rounds starting from the previously mentioned starting index, ensuring that no reconfiguration requests are present between the rounds.
  – A predefined maximum batch size limit.
  – The fetched request is a reconfiguration request, which must be the last request in the batch.

  When a request is added to a batch it is moved to the transaction consensus map and will have the state **AWAITING_PROPOSAL**.

3.1.2 Prepare and Accept

After the preparation phase, the leader will send a single multi-round phase one prepare request to all acceptors for rounds $i+n$, where i is the previously mentioned starting index and $n = 0, \ldots, batch\_size - 1$. This process continues until a majority of acceptors respond with prepare (phase one accepted) for all rounds in the batch. To determine which values to propose in phase two, the leader processes all responses and selects the response value with the highest timestamp. If no value is read from an acceptors response for a given round, the leader will use the value present in the batch for that round. When a majority of acceptors have accepted the phase two proposal for every round in the request, all the proposed transactions are moved to the transaction execution log and their state is now set to **PENDING_EXECUTION** in the transaction consensus map. All the requests present in the batch that didn't end up being proposed are also returned to the queue to be included in future rounds.

3.2. Acceptor

The implementation of the acceptor logic primarily resides within the Paxos gRPC service, where the phase one and phase two remote procedure calls are defined. Acceptors will receive multi-round requests for both the prepare and accept phases, as described in the leader section. They will iterate through these requests and respond with either accept or reject based on the leader's timestamp compared to the highest timestamp they have previously observed and stored for each consensus round. Phase one response values are determined by checking if the request is already present in the log. If not, and there are no uncommitted accepted requests in the round state, the acceptor will return -1 to indicate that no value has been observed for that round. It is important to note that for phase two, we ensure acceptors only send learn requests and register the proposed values in the round state for future reference in case all the rounds in the request have been accepted.

3.3. Learning Phase

The learning phase is responsible for ensuring that all servers, including the newly elected leader, have a consistent view of the transactions that have reached consensus. This is crucial to maintaining consistency across server states in a distributed system. The `LearnHandler` class implements the learning phase logic.

3.3.1 Handling Learn Requests

In order to manage learn requests efficiently and ensure consistency across all server nodes, we maintain a `HashMap` data structure, where each entry is keyed by the consensus round index. Each entry stores a *learn timestamp* and a counter indicating the number of learn messages received for that index.

- **Processing Learn Requests** - When a server receives a `LearnRequest` message, it extracts the index and timestamp from the request. For the corresponding index in the `learnCountMap`, the server checks whether the received timestamp matches the stored learn timestamp. If they are identical, the counter for that index is incremented to reflect the additional learn message was received.
- **Majority Check** - Upon receiving a learn request, the server evaluates if a majority of nodes have reported the same learn timestamp for the given index. When the counter for an index reaches a predefined majority threshold, the server acknowledges that the transaction has reached consensus for that index.
- **Updating Transaction Execution Log** - Once a majority is reached, the server adds the transaction to the transaction execution log at the corresponding index. The status of the transaction is then updated to `PENDING_EXECUTION` in the `Transaction Consensus Map`.
- **Notifying the Executor** - After updating the transaction status, the server signals the execution thread to indicate that a new transaction is available for processing.

3.4. Conclusions

In this project, we successfully implemented a distributed, transactional key-value store using variations of the Paxos consensus algorithm. Our approach allowed us to build a robust system that supports concurrent client transactions while maintaining consistency across multiple server nodes.

The core data structures and strategies chosen—such as the use of thread-safe collections and priority queues—ensured safe and efficient state management during consensus and transaction handling. Throughout the project, key design decisions were guided by the foundational principles of the Paxos algorithm, as described by [1] [2][3]

References

[1] T. D. Chandra, R. Griesemer, and J. Redstone. Paxos made live: An engineering perspective. In *Proceedings of the Twenty-Sixth Annual ACM Symposium on Principles of Distributed Computing (PODC)*. ACM, 2007.

[2] L. Lamport. Paxos made simple. *ACM SIGACT News*, 2001.
[3] L. Lamport, D. Malkhi, and L. Zhou. Reconfiguring a state machine. *ACM SIGACT News*, 2010.